H2H Harness-to-Harness: A Protocol for Local Delegation Among Coding-Agent Harnesses

Contents

Contents 1
Abstract 3
1. Introduction 3
2. Related Work 4
2.1 Harness engineering 4
2.2 Model Context Protocol 4
2.3 Agent2Agent Protocol 4
2.4 Agent Client Protocol 5
2.5 External multi-harness orchestration 5
2.6 Identified gap 5
3. Terminology and Delegation Model 5
4. Methodology 6
4.1 Investigation design 6
4.2 Unit of analysis 7
4.3 Success criterion 7
4.4 Nature of the study 7
5. Proposed H2H v0.1 Protocol 7
5.1 Scope 7
5.2 Harness manifest 8
5.3 Task descriptor 8
5.4 Shared autonomy vocabulary 9
5.5 Guard against recursion 9
5.6 Delegate permission sovereignty 9

5.7 Timeout, output, and resumption 9
5.8 Using ACP as transport 10
6. Case Studies 10
6.1 Closing a categorical gap: image generation 10
6.2 Delegating judgment: editorial and analytical work 11
6.3 Triangulating a real defect 11
6.4 Cross-cutting findings 12
7. Discussion 13
7.1 Three value modes 13
7.2 Relation to existing protocols 13
7.3 Security and accountability 13
7.4 Practical reproducibility 14
8. Limitations and Threats to Validity 14
9. Future Work 14
10. Conclusion 15
References 15

Authors: Jair Lima; Jackson Mallmann
Article version: 1.1
Date: July 31, 2026

About the authors: Jair Lima has worked as a programmer since 1993 and conducts independent research and tool/protocol development. He holds a degree in Computer Networking from Instituto Federal Catarinense (2014), specializations in IT Governance from Centro Universitário Leonardo da Vinci (2015) and in Google for Education Technologies from Universidade LaSalle (2019), and the Chronicle Security Operations Platform – Deep Dive certification (Google Cloud Skills Boost, 2024), in addition to CCNA (Cisco), Windows Server (Microsoft), Educator (Google), and Expert (Adobe) certifications. Contact: jairslima@gmail.com.

Dr. Jackson Mallmann is a professor at Instituto Federal Catarinense (IFC), where he has worked since 2012 in teaching, research, and technological development in the fields of Computer Networks, Computer Security, Cybersecurity, and Artificial Intelligence. He holds a Ph.D. in Computer Science from the Pontifical Catholic University of Paraná (2021), an M.Sc. in Computer Science from the same institution (2011), and a B.Sc. in Computer Science from Santa Catarina State University (2004). His additional training includes Cisco CyberOps Associate and Network Security, Google Cloud Computing Foundations, cloud security and identity, digital forensics, data protection, machine learning, and deep learning. He is the author of scientific articles, books, and technical book chapters, coordinates research on intrusion-detection systems and AI-based ransomware detection, and holds a patent and software registrations related to automated video-content detection. Lattes CV: http://lattes.cnpq.br/4046837503511326.

Abstract

Coding-agent harnesses combine language models, tools, memory, permission policies, and verification mechanisms into complete operational environments. Although protocols such as MCP, A2A, and ACP have advanced interoperability among applications, tools, and agents, one specific scenario has not yet been systematically addressed: a coding harness autonomously invoking another complete harness as a local subprocess, delegating part of a task, and receiving verifiable outputs and artifacts without relying on an external orchestrator. This article names that pattern **H2H, Harness-to-Harness**, and presents a minimal protocol proposal derived from an intensive exploratory investigation conducted over a few days on Windows 11. The corpus included eight tools or execution surfaces and six main empirical cases, five involving production tasks and one serving as a synthetic capability test. The results indicate three recurring value modes: bridging categorical capability gaps, obtaining independent judgment, and triangulating diagnoses in difficult problems. The investigation also identified operational risks, including recursive invocation, excessive autonomy, unenforced timeouts, diagnostic loss caused by truncated output, undeclared external dependencies, and command-line interface drift. In response, H2H v0.1 proposes capability manifests, task descriptors, a shared autonomy vocabulary, a maximum delegation depth, permission sovereignty for the delegated harness, complete capture of output on failure, and artifact-state verification before the caller resumes the task. This work should be interpreted as an exploratory technical report and protocol proposal rather than a controlled performance benchmark.

Keywords: coding agents; harness engineering; agent delegation; interoperability; local automation; multi-agent systems.

1. Introduction

The performance of a coding agent does not depend on the language model alone. It results from the interaction between model, tools, memory, context, permissions, observability, execution mechanisms, and completion criteria. This operational bundle is often described as a *harness* (Zhong & Zhu, 2026; Chen et al., 2026).

In practice, different harnesses offer distinct combinations of models, native tools, security policies, integrations, and non-interactive modes. Even when a product allows selecting more than one model or provider, its tools, skills, memory, execution interface, and authorization policies remain part of the harness. Substituting only the model, therefore, is not equivalent to transferring the full operational capability of one environment to another.

Several harnesses already offer programmatic command-line execution, with input via argument or `stdin` and textual or structured output (Anthropic, 2026; Google, 2026; GitHub, 2026). This makes it technically possible for one process to invoke another and treat its result as part of a larger flow. However, the mere existence of headless commands does not resolve questions of capability discovery, autonomy, recursion, timeout, artifact integrity, external dependencies, and accountability.

This article proposes the concept of **H2H, Harness-to-Harness**, defined as local delegation in which a calling harness, by its own decision or by previously authorized policy, invokes another complete

harness as a subprocess, transmits a task descriptor, receives output and artifacts, and retains responsibility for coordinating the root task.

The investigation was guided by four questions:

1. In which situations does delegation between harnesses produce value that a single harness does not offer as readily?
2. Which operational failures recur when a harness invokes another locally?
3. Which minimal elements should compose a safe and verifiable H2H protocol?
4. How does H2H relate to existing protocols without duplicating their goals?

The main contributions are: the definition of the pattern; its distinction from MCP, A2A, ACP, and external control planes; a minimal specification derived from empirical observations; and the organization of results into three value modes — capability, judgment, and diagnostic triangulation.

2. Related Work

2.1 Harness engineering

Recent harness-engineering literature investigates how memory, tools, observability, flow control, evaluation, permissions, and context structures influence agent performance (Zhong & Zhu, 2026; Wang et al., 2026; Chen et al., 2026; Pan et al., 2026; Sayed Ali et al., 2026). These works treat the harness as a technical object in its own right, not as a mere wrapper around the model.

H2H shares this premise but shifts the focus. Instead of examining only the internal architecture of a single harness, it investigates the operational composition between independent, complete harnesses.

2.2 Model Context Protocol

The Model Context Protocol (MCP) standardizes the connection between host applications and servers that expose tools, resources, and prompts. Its architecture favors composing specialized capabilities and can operate over local `stdio` or HTTP transport (Model Context Protocol, 2025).

An MCP server, however, need not be a complete coding harness. In general, it exposes focused capabilities under the control of a host application. H2H addresses a different unit of composition: a complete agent environment, with its own model, tools, memory, policies, and execution cycle, delegated as an operational black box.

2.3 Agent2Agent Protocol

The Agent2Agent Protocol (A2A) standardizes communication between independent, potentially opaque agent systems. The protocol defines discovery via Agent Cards, messages, tasks, artifacts, and network transports (A2A Project, 2026). Its scope is broad enough to represent collaboration between agents and could, in principle, be used on a single local machine via a network endpoint.

The central difference is not that A2A is incapable of describing delegation, but that H2H pursues a smaller solution oriented toward local subprocesses, without requiring every harness to be published as a service. H2H also makes normative several concerns that arose from running local CLIs in practice: delegation depth, mapping of autonomy flags, capture of `stdout` and `stderr`, working directory, caller-imposed timeout, and verification of partial changes.

2.4 Agent Client Protocol

The Agent Client Protocol (ACP) standardizes sessions between clients and coding agents. In its local mode, the client initializes the agent as a subprocess and communication occurs via JSON-RPC over `stdin` and `stdout`; the ecosystem is also working on stabilizing remote transports (Agent Client Protocol, 2026a; 2026b).

ACP is especially close to H2H at the transport level. It already provides initialization, sessions, streaming, and permission requests. However, ACP does not by itself define the policy by which a harness decides to delegate to another, nor does it establish the specific set of safeguards proposed in this work. Accordingly, an H2H implementation may use ACP when the delegated harness supports it, falling back to the native CLI otherwise.

2.5 External multi-harness orchestration

Control planes such as Warp's Oz execute and coordinate multiple harnesses from an external layer, with observability, governance, and parallelism (Warp, 2026). This model is valuable but differs from H2H in its decision point. In a control plane, a higher-level system coordinates the agents; in H2H, the decision to delegate can arise inside the calling harness itself, during the task.

2.6 Identified gap

In the literature and technical specifications reviewed, no minimal protocol was identified that is dedicated to the case in which a coding harness, running locally, invokes another complete harness as a subprocess, retains coordination of the root task, and applies normative safeguards for recursion, autonomy, timeout, and artifact integrity.

This framing is deliberately narrow. H2H does not intend to replace general agent-communication protocols, nor external orchestration systems. It seeks to standardize a local practice that can already be performed by scripts, but which remains fragmented and dependent on ad hoc, individual conventions.

Recent work approaches adjacent aspects of the problem from complementary perspectives. Recursive Agent Harnesses studies executable processes that spawn other agent harnesses, emphasizing recursion and parallelism. SentinelAgent formalizes verifiable delegation chains and policy preservation in multi-agent systems. Code as Agent Harness examines code as executable infrastructure for agent coordination and verification. H2H differs from these approaches by proposing a minimal contract for local, synchronous, depth-1 delegation between complete harnesses, using either a native CLI or ACP while preserving the delegated harness's policy sovereignty.

3. Terminology and Delegation Model

For the purposes of this article, the following definitions are adopted:

Term	Definition
Coding harness	Operational environment combining one or more models, tools, memory, context, permissions, execution, and verification of software or technical-production tasks.
Calling	Harness that retains responsibility for the root task and initiates delegation.

harness	
Delegated harness	Harness invoked to execute a bounded subtask.
Harness manifest	Declarative document describing invocation method, capabilities, autonomy levels, dependencies, and known constraints.
Task descriptor	Structure transmitted to the delegated harness with objective, context, working directory, autonomy, timeout, output format, and relevant history.
Delegation depth	Number of consecutive H2H transitions within the same root task.
Target artifact	File, directory, branch, image, report, or other object that may be created or modified by the delegated task.

For brevity, the calling harness and the delegated harness are hereafter also referred to as the **caller** and the **delegate**, respectively; both forms are used interchangeably throughout this article.

The minimal model has four steps:

1. The caller identifies a subtask and selects a compatible delegate.
2. The caller consults the manifest and prepares the task descriptor.
3. The delegate executes the subtask under its own security policies.
4. The caller validates the output, verifies the artifacts, and decides to accept, reject, complement, or delegate again to another harness, subject to the maximum depth.

The delegated harness is treated as a black box with respect to internal reasoning, but not with respect to observable effects. Commands executed, files modified, outputs, return codes, and checks should remain auditable whenever the tool makes them available.

4. Methodology

4.1 Investigation design

The study adopted an exploratory, inductive approach. Rather than defining a complete specification up front, delegation tests were run, and the requirements of H2H v0.1 were derived from the failures and regularities observed.

The investigation was conducted over an intensive four-day period, from July 26 to July 29, 2026, on a Windows 11 development machine. Eight harnesses, CLIs, or control surfaces were examined: Claude Code, Codex CLI, Gemini CLI, Antigravity/`agy`, GitHub Copilot CLI, OpenCode, Cursor Agent CLI, and Visual Studio Code. VS Code served as a conceptual negative control, since it is an editing and integration surface but does not by itself constitute a headless delegated harness equivalent to the others.

Tool versions changed during the period. This lack of a frozen baseline does not allow a controlled comparison across builds, but it made a relevant operational problem visible: drift in flags, commands, and capabilities across updates.

4.2 Unit of analysis

The unit of analysis was a delegated execution with a verifiable objective. For each execution, the following were recorded when available:

- calling harness and delegated harness;
- command or invocation mode;
- working directory;
- task objective;
- requested permission policy;
- observed result;
- files or diffs produced;
- security blocks;
- need for human interruption;
- subsequent verification of the artifact.

The main set comprised six empirical executions: one synthetic image-generation proof and five production tasks — two book covers, one editorial review, one citation correction, and the investigation of a software defect involving three successive delegated attempts.

4.3 Success criterion

An execution was not considered successful merely because the harness declared the task complete. The result had to be confirmed by at least one of the following means:

- direct visual inspection of the produced file;
- full reading of the generated report;
- checking the diff;
- running the project's own tests or verifiers;
- re-exporting and validating the application;
- confirming behavior on real hardware.

4.4 Nature of the study

The study does not measure which harness is "better." Cases were not balanced by model, version, cost, number of attempts, or complexity. The goal is to identify composition patterns, failures, and safeguards. The results should therefore be interpreted as evidence of feasibility and as a basis for generating requirements, not as a product ranking.

5. Proposed H2H v0.1 Protocol

5.1 Scope

Version 0.1 is limited to delegation that is:

- local, on the same machine;
- synchronous from the root task's point of view, even though the subprocess may be monitored asynchronously;
- initiated via subprocess, native CLI, or a local ACP session;
- of maximum depth 1;

- between one caller and one delegate per subtask.

Out of scope for this initial version: multiple hops, delegation distributed across machines, multi-user environments, consensus among agents, parallel fan-out execution, and automatic arbitration between conflicting results.

5.2 Harness manifest

Each harness must have a verifiable manifest containing, at minimum:

- identifier and detected version;
- official installation source;
- executable or ACP endpoint;
- non-interactive invocation mode;
- working-directory parameter;
- input and output formats;
- known return codes;
- declared capabilities;
- supported autonomy levels;
- known external dependencies;
- platform constraints;
- health check command;
- date of the manifest's last validation.

The manifest must not rely on third-party packages with names similar to official products without explicit verification of origin and maintainer.

5.3 Task descriptor

The task descriptor should separate objective, permissions, and context. A minimal format can be represented as follows:

```
{
  "h2h_version": "0.1",
  "task_id": "uuid",
  "caller": "harness-a",
  "delegate": "harness-b",
  "objective": "bounded description of the subtask",
  "working_directory": "absolute path",
  "target_artifacts": ["file-or-directory"],
  "autonomy_requested": 1,
  "timeout_seconds": 900,
  "output_format": "json",
  "previous_attempts": [],
  "delegation_depth": 1
}
```

The `previous_attempts` field should be optional and contain only useful information: harness used, hypothesis, change applied, and observed result. This history reduces repetition and allows a new delegate to look for the common pattern across previous attempts.

5.4 Shared autonomy vocabulary

Harnesses use incompatible naming conventions for permissions. H2H proposes four reference levels:

Level	Name	Operational limit
0	Advisory	Reading and analysis only; no artifact modification and no actions with persistent effects.
1	Controlled editing	May modify only the target artifacts or the designated workspace; other actions require approval.
2	Workspace autonomy	May edit, run commands, tests, and verifiers within the workspace, without administrative privileges or global changes.
3	Extended autonomy	May request broader actions, including network access or dependency installation, but remains subject to the delegated harness's own policies and blocks.

The manifest must map these levels onto the native flags available. When there is no exact equivalence, the delegate should apply the safer, lower level and report the downgrade.

5.5 Guard against recursion

v0.1 establishes two mandatory rules:

1. A harness may not invoke itself within the same root task.
2. A harness that has received a task via H2H may not perform a new H2H delegation within that same task.

A maximum depth of 1 reduces cycles, cost explosion, loss of attribution, and repeated external side effects. Work on agentic loops shows that flows without effective limits can amplify calls, cost, and side effects (Hou et al., 2026).

5.6 Delegate permission sovereignty

The caller may request an autonomy level, but cannot force the delegate to exceed its own policies. The effective level is determined by the minimum of:

- autonomy requested by the caller;
- autonomy allowed by the user or environment;
- autonomy supported by the delegated harness;
- additional sandbox or operating-system constraints.

Natural-language authorizations should not substitute for formal permission mechanisms. When a dangerous flag is blocked by the delegate, H2H should log the block and either proceed at a lower level or terminate the task.

5.7 Timeout, output, and resumption

The `timeout_seconds` value must be enforced by the caller. Including it in the prompt or descriptor alone is not sufficient. Upon expiry, the caller should:

1. request graceful termination, when possible;
2. kill the process after a short grace period;
3. capture `stdout`, `stderr`, exit code, and final state;
4. check whether the target artifacts were modified;
5. prevent a new attempt from overwriting unevaluated partial changes.

On failure, output must not be truncated in the technical record. A summarized interface may show only the essentials to the user, but the complete log needs to be preserved for diagnosis.

5.8 Using ACP as transport

When the delegate offers an ACP server, an H2H implementation may prefer it over ad hoc text capture from the CLI. ACP provides structured communication, streaming, sessions, and permission requests. Even so, the rules for delegate selection, depth, timeout, permission sovereignty, and artifact integrity remain the responsibility of the H2H layer.

6. Case Studies

6.1 Closing a categorical gap: image generation

6.1.1 Synthetic capability proof

In the tested configuration, Claude Code exposed no native image-generation tool. The calling harness delegated to the Codex CLI the creation of a red apple on a white background. The delegated environment selected an image-generation capability and returned a PNG file consistent with the request, confirmed by visual inspection.

The value of this case does not lie in the difficulty of the image. It demonstrates a categorical gap: the caller lacked the necessary tool, while the delegate had it. Delegation avoided any attempt to simulate the missing capability or to substitute only the model inside the calling harness.

6.1.2 Cover for the book on RISC-V

The pattern was applied to a production book cover. The background art was generated with no embedded text. A deterministic PowerShell script, using `System.Drawing`, then applied title, subtitle, tagline, and author name using system fonts.

The final file measured 1600 × 2560 pixels with 300 DPI metadata. This hybrid composition removed the image model's involvement in reproducing accents and diacritics. Textual fidelity became deterministic, limited only by font availability, typographic parameters, and the rendering mechanism.

This workflow also produced a reusable and reproducible artifact: the composition script itself. As a result, text or position changes can be redone without regenerating the background image.

6.1.3 Cover for the book on CRISPR gene editing

For a second cover, an automation wrapper terminated without producing `capa.png` and without surfacing the cause in the summary it displayed. Invoking the delegate directly, with full `stdout`/`stderr` capture, revealed that one pipeline step attempted to use ImageMagick, which was absent from the machine.

The delegate replaced that step with a PowerShell/`System.Drawing` solution and completed the cover. This observation does not license the claim that the entire image skill depends on ImageMagick. What was demonstrated is that the pipeline executed in that attempt had an external dependency that was unmet and previously undeclared.

Two rules were derived:

- wrappers should not hide complete output when execution fails;
- manifests should declare external executables used by skills, scripts, or dynamically generated steps.

6.2 Delegating judgment: editorial and analytical work

6.2.1 Structured editorial review

Codex was assigned the role of professional editor to assess a book's draft. The instruction required each finding to be classified by the strength of intervention it required.

The result was an approximately 23 KB editorial assessment, read in full, with an executive summary, strengths, and 17 issues. Four were classified as `SIMPLE CORRECTION` and thirteen as `STRONG STRUCTURAL/EDITORIAL CHANGE`.

The main outcome was not merely the content of the review, but its decision structure. Self-classification by impact allowed potentially automatable interventions to be separated from decisions that should remain with the author. This pattern transfers the same logic used for operational autonomy levels onto the analytical output itself.

6.2.2 Citation correction under an explicit anti-fabrication rule

In another manuscript, twelve names appeared with inconsistent citations or insufficient bibliographic entries. The delegated task established a conservative rule:

- use the author-date system with years already supported by the bibliography;
- never invent a page number;
- search for a real source when a reference was missing;
- remove the named attribution and generalize the statement when no secure confirmation was found.

Verification occurred via diff and via the project's own tools. Twenty-seven citations were reviewed; twenty-two were classified as strong by the internal verifier; none remained unlocated; and the final analysis recorded no critical issue. The delegate located sources with a DOI for two authors, removed two names lacking secure support, and identified one additional incorrect attribution.

The case suggests that explicit, conservative, verifiable constraints can be delegated more safely than generic instructions such as "fix the references." The operative rule was not "find an answer," but "do not fabricate when the evidence is insufficient."

6.3 Triangulating a real defect

6.3.1 Observed problem

A 3D screensaver built in Godot worked correctly on the primary monitor but showed a black screen on the secondary monitor.

6.3.2 First attempt: Codex CLI

Codex read the code and consulted documentation on multiple windows. Execution, however, remained stalled for more than fifteen minutes with no perceptible progress. A stalled network-access step within the default sandbox was suspected, although this could not be confirmed.

Because the caller did not enforce an automatic timeout, the interruption depended on a human decision. Before resuming the work, it was verified that no file had been altered. The case demonstrated that documenting a timeout without enforcing it does not provide effective control.

6.3.3 Second attempt: Antigravity/`agy`

The second delegate proposed that cameras and textures attached directly to a native `Window` created at runtime might not have a reliable drawing pipeline in Godot 4. The recommendation was to adopt `SubViewportContainer` and `SubViewport` with a shared `world_3d`.

The change was implemented, validated for errors, and re-exported. The defect persisted. The hypothesis was technically plausible but did not explain the behavior observed on the hardware.

6.3.4 Third attempt: GitHub Copilot CLI

The third delegation included the history of previous attempts. Copilot observed that both shared a common element: creating a second native Windows window. The proposed hypothesis was a failure in creating or presenting a second Vulkan swapchain when the monitors were connected to different graphics outputs.

The applied solution consolidated rendering into a single window, with one `SubViewport` region per monitor. The project was re-exported, and the user confirmed it worked on real hardware.

The success of the fix supports the second-swapchain hypothesis but does not conclusively prove it, since no driver logs or Vulkan traces were collected. The accurate statement is: the third diagnosis produced the most consistent operational explanation and the only fix confirmed in the real environment.

6.3.5 Derived lessons

The case supports four requirements:

1. consulting different harnesses can reveal complementary hypotheses;
2. the history of attempts should accompany new delegations;
3. timeout needs to be enforced by the caller;
4. artifact state must be verified before resumption.

A supervision lesson also emerged. During long-running tasks, duplicate monitors were created out of impatience, even though the first one remained active. The derived operational rule is simple: a task should have a single responsible monitor, identified by `task_id`.

6.4 Cross-cutting findings

Frictionless autonomy. In the observed matrix, the combination of Claude Code as caller and GitHub Copilot CLI as delegate was the only one that accepted the requested autonomy level without triggering an additional safety block. This result is specific to the versions and configurations tested and should not be generalized as a permanent property of the products.

Block consistent with recursion protection. The attempt to use the Codex CLI to invoke Claude Code was blocked by a protection consistent with a nested or recursive session guard. The result reinforced the adoption of a maximum depth of 1.

Supply-chain risk. In tests related to Antigravity and Cursor Agent CLI, third-party packages or repositories were found with names very similar to the official products. None was installed. The H2H manifest should record only official sources, or sources whose provenance has been explicitly verified.

Manifest drift. An update to the Codex CLI removed a flag used in the documented delegation command. No warning was noticed in the local flow, and the failure was only identified on a subsequent run. This case motivated health checks that compare the current `--help` output against the known manifest.

7. Discussion

7.1 Three value modes

The cases point to three independent value modes:

1. **Capability:** the delegate has a tool or integration absent from the caller.
2. **Judgment:** the delegate offers an independent reading, a specialty, or a structured critique.
3. **Triangulation:** different harnesses formulate distinct hypotheses for a problem that resists a single line of investigation.

This classification avoids reducing H2H to a simple "model swap." In the first mode, the advantage may lie in a native tool. In the second, it lies in the independence of judgment. In the third, it lies in the diversity of diagnostic paths and in the ability to incorporate the history of attempts.

7.2 Relation to existing protocols

MCP can expose specialized tools to the caller. A2A can connect independent agents via network services. ACP can provide structured transport between client and agent. Control planes can coordinate fleets of agents and parallel executions.

H2H operates at a narrower layer: it defines how a local harness decides on, and controls, the invocation of another complete harness, especially when the only common ground available is a headless CLI. Its contribution lies not in creating a new universal message format, but in formalizing operational requirements that can be applied over a native CLI or over ACP.

7.3 Security and accountability

Delegation does not fully transfer responsibility. The caller remains responsible for:

- choosing the delegate;
- bounding the objective;
- requesting proportionate autonomy;
- enforcing timeout;
- preserving logs;
- validating artifacts;

- preventing recursion;
- deciding whether the result will be incorporated.

The delegate remains sovereign over its own permission policies. This division avoids two extremes: a caller unable to control the effects of the subprocess, and a caller able to unilaterally disable all of the delegate's protections.

7.4 Practical reproducibility

The main threat to reproducibility is the rapid evolution of CLIs. Commands, flags, models, and authentication change frequently. Reproducing H2H therefore requires more than recording the prompt. It is necessary to record the executable's version, the `--help` output, the operating system, the working directory, the permission configuration, external dependencies, and exit codes.

The manifest should be treated as a versioned, testable artifact, not as static documentation.

8. Limitations and Threats to Validity

This study has significant limitations:

- it was conducted by the authors, on a single machine, and on a single operating system;
- the observation period was short and intensive (a few days), not longitudinal; this favored the concentrated collection of real production cases, but it does not allow assessment of harness stability, failure recurrence, or manifest drift over time — only what was observed within the window studied;
- even within this short window, harness versions did not remain frozen (at least two CLI updates were observed within a few days), which already signals that interface instability tends to be more severe, not less, over longer windows;
- tasks were not evenly distributed across all products;
- some of the systems are proprietary and opaque, limiting inspection of the internal cause of failures;
- the production cases belong to the authors' own workflow and may not represent other domains;
- there was no independent replication;
- some causes were inferred from behavior and a successful functional fix, without low-level telemetry;
- the study does not control for differences in model, context, cost, latency, or number of attempts.

These limitations preclude comparative conclusions about the overall quality of the harnesses. They do not, however, eliminate the usefulness of the cases for revealing failure classes and generating protocol requirements. Future validation should include replication on Linux and macOS, frozen versions, a reproducible delegation matrix, and publication of sanitized logs and artifacts.

9. Future Work

The evolution toward H2H v0.2 should prioritize:

1. a reference implementation with real timeout enforcement and safe termination;

2. a machine-validatable JSON schema for manifests and task descriptors;
3. an automatic health check to detect CLI drift;
4. support for attempt history and correlation via `task_id`;
5. centralized logging of cost, tokens, duration, and artifacts;
6. ACP adapters for compatible harnesses, with native-CLI fallback;
7. a routing policy based on capability, risk, and budget;
8. a public test suite with synthetic tasks and sanitized production cases;
9. a subsequent study of fan-out and depth greater than 1, only after formal cycle-prevention mechanisms are in place.

10. Conclusion

This article defined H2H as a local delegation pattern in which a coding-agent harness invokes another complete harness as a subprocess, without relying on an external orchestrator. The proposal emerged from a short, intensive period of empirical observation (a few days) and six main executions, five in production and one synthetic.

The cases demonstrated three forms of value: closing capability gaps, obtaining independent judgment, and triangulating diagnoses. They also showed that composing harnesses introduces risks of its own — notably recursion, poorly mapped autonomy, unenforced timeout, diagnostic loss, undeclared external dependencies, and interface drift.

H2H v0.1 responds with a deliberately small core: harness manifest, task descriptor, four autonomy levels, a maximum depth of 1, delegate permission sovereignty, complete capture of output on failure, and artifact verification before resumption.

The main conclusion is that harnesses need not be treated as isolated environments, nor reduced to the models they use. They can become compositional units. For that composition to be trustworthy, however, delegation must stop being an improvised script and instead acquire a contract, boundaries, observability, and explicit accountability.

References

A2A Project. (2026). *Agent2Agent protocol specification*. Linux Foundation. https://a2aproject.github.io/A2A/latest/specification/

Agent Client Protocol. (2026a). *Architecture*. https://agentclientprotocol.com/get-started/architecture

Agent Client Protocol. (2026b). *Introduction*. https://agentclientprotocol.com/get-started/introduction

Anthropic. (2026). *Claude Code CLI reference*. https://code.claude.com/docs/en/cli-usage

Chen, T., Lu, S., Zhao, K., Meng, W., Teng, H., Li, T., Li, C., Liu, X., Liang, J., Zhang, Z., Xie, Y., Qu, H., Shao, K., & Luan, J. (2026). *HarnessX: A composable, adaptive, and evolvable agent harness foundry*. arXiv. https://arxiv.org/abs/2606.14249

GitHub. (2026). *GitHub Copilot CLI programmatic reference*. https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-programmatic-reference

Google. (2026). *Gemini CLI: headless mode reference*. https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/headless.md

Hou, X., Wang, S., Zhao, Y., & Wang, H. (2026). *When agents do not stop: Uncovering infinite agentic loops in LLM agents*. arXiv. https://arxiv.org/abs/2607.01641

Model Context Protocol. (2025). *Architecture (Protocol revision 2025-06-18).* https://modelcontextprotocol.io/specification/2025-06-18/architecture

Pan, L., Zou, L., Guo, S., Ni, J., & Zheng, H.-T. (2026). *Natural-language agent harnesses*. arXiv. https://arxiv.org/abs/2603.25723

Sayed Ali, M., Novik, A., Boddupally, A., Yavorskyi, A., Nickerson, C., Rica, D., DuGranrut, E., Leung, F., Prince, G., Barnett, G., Robinson, H., Al Ahmad, H., Resnick, J., Farah, J. C., Meruga, J. S., Kuznetsov, L., Perry, B., Gorham, L., Schmoll, M., … AlShikh, W. (2026). *The harness effect: How orchestration design sets the token economics of enterprise agentic AI*. arXiv. https://arxiv.org/abs/2607.06906

Wang, R., Shi, Y., Li, Z., Li, Z., Yu, Y., Yang, J., Panaganti, K., Mi, H., Zhou, D., & Leoweiliang. (2026). *Harness handbook: Making evolving agent harnesses readable, navigable, and editable*. arXiv. https://arxiv.org/abs/2607.13285

Warp. (2026, May 19). *A single pane of glass for managing all of your cloud agents*. https://www.warp.dev/blog/multi-harness-cloud-agent-orchestration

Zhong, H., & Zhu, S. (2026). *AI harness engineering: A runtime substrate for foundation-model software agents*. arXiv. https://arxiv.org/abs/2605.13357